Computational Complexity

Reading guide and Homework for **Week 09**

1 Reading Guide

This week:

- We learned about probabilistic algorithms, and defined the three classes $\mathsf{BPP}, \mathsf{RP}$ and ZPP. This is Section 7.1-7.4 of the book.

2 Homework

Skill: *Be thorough by finishing the details we ommitted in class.*

1. Check that the recursively-defined $T(w, n)$ is $O(wn)$.

2. (Amplification of success probability by repitition) Suppose that μ is a distribution with $\mu(1) \geq 2/3$ and $\mu(0) = 1 - \mu(1)$. Let $X_1, \ldots, X_m$ be m independent and identically distributed (iid) random variables, each distributed according to μ. Show that by choosing $m = O(n)$ sufficiently large, we have:

$$\Pr[\sum_i X_i \leq \frac{m}{2}] \leq 2^{-n}.$$

   Now apply this to the case where μ gives us the probability that a certain randomized algorithm A answers correctly on a certain input x. Understand: it follows that, if we run $A(x)$ m times, each with independent randomness, and output the majority answer, then the probability that our output is wrong is $\leq 2^{-n}$.

3. Prove that, if we have an algorithm A that outputs $f(x)$ with probability slightly larger than $1/2$, say $1/2 + \frac{1}{n^c}$, then we can again repeat the algorithm and take majority answer and this will decrease the probability of failure down to inverse-exponential. How many repititions are necessary, in this case?

4. Prove that, if we have a one-sided-error algorithm A that outputs $f(x)$, always correctly when $f(x) = 0$, and with probability at least $\frac{1}{n^c}$ when $f(x) = 1$, then we can again repeat the algorithm and this will decrease the probability of failure down to inverse-exponential. How many repititions are necessary, in this case? What should we do with the outputs of the repeated runs? (It is not taking the majority answer. What makes sense?)

5. Carefully prove that $\mathsf{ZPP} = \mathsf{RP} \cap \mathsf{coRP} \subseteq \mathsf{NP}$.

Skill: *Reason about probabilistic algorithms.*

6. Show that $\mathsf{BPP} \subseteq \mathsf{PSPACE}$.

7. Imagine we generalise the definition of BPP by allowing the PTM to run in expected time polynomial in the input size. That is, the expected running time of the PTM is polynomial on every input (but sometimes, depending on its internal randomness, it might run for much longer). Does this change the definition of BPP?

8. Now imagine we generalise the definition of BPP so that the algorithm only needs to be correct with probability strictly greater than 1/2. (We call this class PP.) Show that such algorithms can decide all of NP.

9. Do exercises 7.1 and 7.6.

Skill: *Learn things on your own.*

10. Read and learn the proof that $\mathsf{BPP} \subseteq \Sigma_2 \cap \Pi_2$ (Section 7.5.2 of the book).